

Cloud & Local Recovery via 2-of-3 Shamir DEK Sharding

Status: PLANNED (design spec). Not implemented. **Scope:** geographic resilience for the seed via Shamir Secret Sharing on the DEK, with device-local, cloud, and physical backup share paths. **Owner:** AI · **Reviewer (required before implementation):** independent audit **Framing:** PRE-AUDIT. This is a design document; no code ships without the audit. **Prerequisite:** vault AAD v:3 migration (#1111) should land first so the share envelope can bind AAD fields from day one.

Design principle — touch the KEK zero. The entire KEK construction (`combineKek` , `H||C` , HKDF, hardware factor, deniability forking) is UNCHANGED. The Shamir split happens to the DEK after generation and before KEK wrapping. Share A is wrapped by the KEK exactly as the full DEK is today. From the KEK's perspective nothing changed — it wraps a 32-byte value and hands it back.

1. The problem this closes

Today the seed is recoverable only from the device that holds the KEK-wrapped DEK. If that device is lost, destroyed, or permanently locked, the seed is gone — unless the user proactively exported a backup envelope (`vaultBackup.js`). Most users never export.

Cloud recovery (TARGET in the roadmap) aims to maintain an off-device recovery path **automatically**, without requiring proactive user action beyond initial setup. Local recovery (NFC/QR/paper) provides the same resilience for users who distrust cloud storage.

Non-goals:

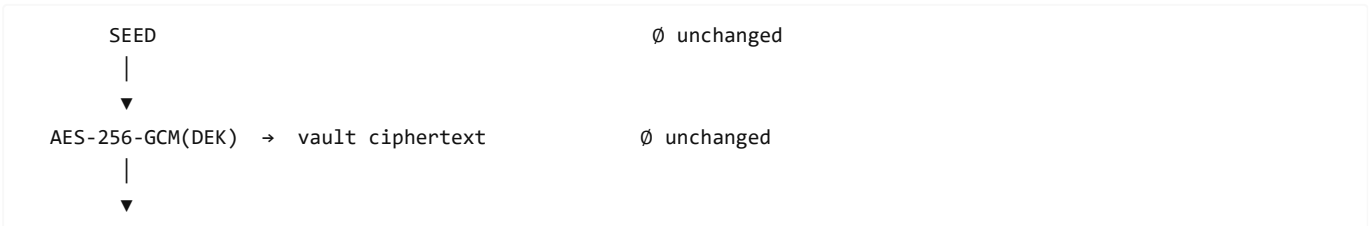
- This spec does NOT change how the seed is encrypted (AES-256-GCM under DEK).
- This spec does NOT change the KEK derivation (`HKDF(H||C)`).
- This spec does NOT introduce custodial key management or server-held keys.
- This spec does NOT replace the existing backup envelope — it complements it.

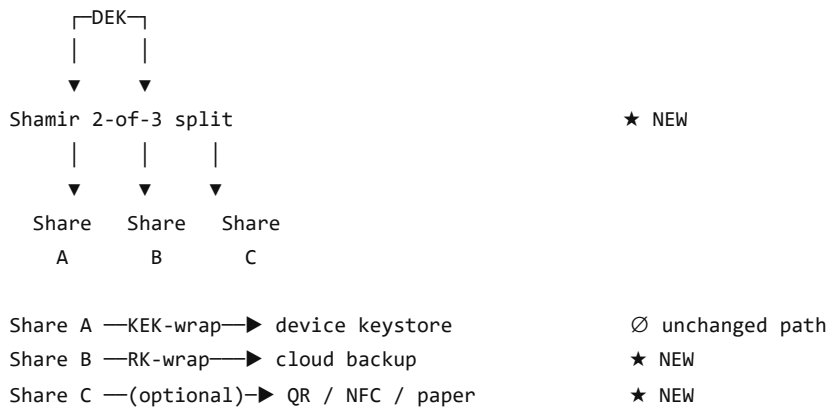
2. Threat model additions

Threat	Mitigation
Device lost/destroyed	Reconstruct DEK from any 2 of 3 shares
Cloud provider compromise	Cloud share alone reveals zero DEK bits (Shamir property)
Physical backup stolen	Single share alone reveals zero DEK bits
Cloud + physical both compromised	DEK reconstructable — equivalent to vault seizure today; still requires vault ciphertext + the reconstructed DEK to reach the seed
Coercion to reveal cloud passphrase	Deniability: decoy DEKs have their own independent shard sets (§7)
Share C physically discoverable	Open question — see §10

Availability regression (honest): the current model needs 1 device + 1 PIN. This model needs 2-of-3 things. Losing any 2 shares permanently destroys the wallet. This is the fundamental tradeoff of geographic resilience in a non-custodial system.

3. Keying stack (unchanged layers marked ∅)





3.1 KEK layer (∅ unchanged)

```

KEK = HKDF-SHA256(
  ikm = H || C,
  salt = KEK_HKDF_SALT,
  info = 'veyrnox/kek/v1/combine(H||C)'
)
  
```

- H: 32-byte hardware factor (iOS Secure Enclave ECIES / Android HMAC-SHA256)
- C: 32-byte Argon2id(PIN, salt_set)
- Both required; all-zero rejected; zero post-use

The KEK wraps Share A via AES-256-GCM (`wrapDek` / `unwrapDek` in `kek.js`). From the KEK's perspective Share A is a 32-byte opaque value — identical to the full DEK today.

3.2 Shamir layer (★ new)

The DEK (32 bytes) is split into 3 shares using Shamir's Secret Sharing over $GF(2^8)$, with threshold $k=2$:

- Any 2 shares reconstruct the DEK exactly.
- Any 1 share alone reveals zero information about the DEK (information-theoretic security, not computational — this is a property of the math, not an assumption about attacker resources).

Each share is 33 bytes: 1 byte x-coordinate (share index, 1–3) + 32 bytes y-values. The x-coordinate is non-secret (it identifies which share this is).

Library: `@noble/shamir` (if available) or a hand-rolled $GF(2^8)$ implementation (~200 lines). The implementation MUST be constant-time on the share bytes to prevent timing side-channels during reconstruction. The same `@noble` / `@secure` audit lineage as the rest of the crypto stack.

RNG: `crypto.getRandomValues` for the random polynomial coefficients — the same CSPRNG used for DEK generation. Never `Math.random`.

4. Share A — Device (existing path, minimal change)

What changes: `wrapDek(kek, dek)` today wraps the full 32-byte DEK. After this spec, it wraps Share A (also 32 bytes of secret material, with the 1-byte index stored alongside as non-secret metadata).

Storage: Keychain / Keystore, same key (`vault_v1` under prefix `veyrnox_`), same `@aparajita/capacitor-secure-storage` path. The vault blob gains one field: `shamirIndex: 1` (the x-coordinate, non-secret).

Unlock path: identical to today. `unwrapDek(kek, wrappedShareA)` → Share A. If only Share A is available (normal case), the app needs a second share to reconstruct the DEK. In steady state the second share comes from the device-cached reconstruction (§6.1) — the user never sees the Shamir layer on normal unlock.

4.1 Steady-state unlock (no user-visible change)

On first successful reconstruction (enrollment or recovery), the full DEK is reconstructed in memory from 2 shares, then:

1. The DEK encrypts/decrypts the seed (unchanged).
2. Share A is KEK-wrapped and stored (unchanged path).
3. The full DEK is also KEK-wrapped and stored under a second key (`vault_dek_v1`) as the **fast-path cache**.

On subsequent unlocks, the fast-path cache is tried first. If it succeeds, the Shamir layer is never invoked. If it fails (KEK changed, cache cleared, panic wipe), the app falls back to 2-share reconstruction.

This means normal unlock is **zero additional latency** — the Shamir split is only exercised at enrollment, recovery, and PIN rotation.

5. Share B — Cloud

5.1 Protection

Share B is double-wrapped before leaving the device:

```
Layer 1 (inner): AES-256-GCM under Recovery Key
Layer 2 (outer): Platform end-to-end encrypted backup

Recovery Key = Argon2id(
    recovery_passphrase,
    random_salt,
    memorySize = 196608,    // 192 MiB — same as vault KDF
    iterations = 3,
    parallelism = 1,
    hashLength = 32
)
```

The recovery passphrase is a user-chosen strong passphrase (minimum 16 characters, enforced at setup — longer than the 12-char PIN floor because this passphrase protects an offline-attackable blob that may persist in cloud backups indefinitely).

The random salt is stored alongside the encrypted share (non-secret, same as vault.js convention).

5.2 Transport

- **iOS:** iCloud Keychain (`kSecAttrSynchronizable = true` , `kSecAttrAccessible = kSecAttrAccessibleAfterFirstUnlock`). Apple encrypts end-to-end with the user's iCloud Keychain escrow key. The inner AES-GCM layer means Apple cannot read the share even if they could read the Keychain entry.
- **Android:** Google Backup (Auto Backup or Key/Value Backup via `BackupAgentHelper`). Google encrypts client-side with a key derived from the device's lock screen credential (Android 9+). The inner AES-GCM layer means Google cannot read the share even if they could decrypt the backup.
- **Neither platform:** if the user has disabled cloud backup or is on a platform without it, Share B is not created. The user is informed that recovery requires Share A + Share C (2-of-2 mode). No silent degradation — the recovery setup screen shows exactly which shares exist and where.

5.3 Cloud share format

```
{
  "app": "veyrnox",
  "type": "recovery-share",
  "version": 1,
  "shareIndex": 2,
```

```
"kdf": { "parallelism": 1, "iterations": 3, "memorySize": 196608, "hashLength": 32 },
"salt": "<base64, 32 bytes>",
"iv": "<base64, 12 bytes>",
"ct": "<base64, encrypted share B>"
}
```

The `shareIndex` is non-secret (it tells the reconstruction which x-coordinate this share carries). The `ct` is AES-256-GCM ciphertext of the 32-byte share under the Recovery Key, with AAD = `{ "app": "veyrnox", "type": "recovery-share", "version": 1, "shareIndex": 2 }`.

6. Share C — Local backup

Share C is the user-exported physical share. It exists for users who want recovery without trusting any cloud provider, or as a third share for maximum resilience.

6.1 Export formats

Format	Encoding	Size	Use case
QR code	Base44 of 33-byte share	~60 chars, fits smallest QR	Print, photograph
NFC tag	Raw NDEF binary record	33 bytes	Tap to write/read
File	.veyrnox-share binary	33 bytes + 8-byte header	USB, AirDrop

All formats carry the same 33 bytes (1 byte index + 32 bytes share). The share is **not encrypted by default** — it is the user's physical responsibility, same as a seed phrase written on paper. Optional passphrase wrapping is available (same Argon2id construction as Share B) for users who want defense-in-depth on the physical copy.

6.2 QR encoding detail

Base44 is already used in the codebase (`base44-inventory.md`). A 33-byte share encodes to ~48 Base44 characters, fitting in a Version 2 QR code (alphanumeric mode, 25×25 modules) — small enough to print on a business card or stamp into metal.

The QR payload is prefixed with `VXRS:` (Veyrnox Recovery Share) for type detection on scan. Full format: `VXRS:<base44 of 33 bytes>`.

6.3 NFC detail

NDEF record type: `application/vnd.veyrnox.recovery-share`. The 33-byte payload is written raw (no encoding overhead). Compatible with any NFC Type 2/4 tag with ≥64 bytes usable capacity (~\$0.10 tags). Read requires NFC permission on the device; write requires physical proximity.

7. Deniability (I3 — zero backend calls in deniability mode)

Each credential set (real, duress, stealth, panic) has its own DEK. Under this spec, each DEK gets its own independent Shamir split with its own shares. The share sets are indistinguishable in size and shape (all 33 bytes, all AES-GCM wrapped with the same KDF params).

- **Real set:** Shares `A_real`, `B_real`, `C_real`
- **Decoy set:** Shares `A_decoy`, `B_decoy`, `C_decoy`
- **Stealth/panic:** their own share sets

An examiner who obtains Share B from the cloud sees a recovery-share blob with no indication of which credential set it belongs to. The share index (1/2/3) is identical across sets. The share VALUES differ (different DEK → different polynomial → different shares), but the values are indistinguishable from random ($GF(2^8)$ output).

Critical rule: recovery setup MUST NOT store any mapping between share sets and credential names. The mapping is implicit: `Share A_real` is the one unwrapped by `KEK(H, C_real)`. An examiner who doesn't know the real PIN cannot identify which cloud share belongs to the real set.

Deniability-mode behavior: in a decoy/demo session, recovery setup and share export operate on the DECOY DEK's shares. No real-set shares are read, written, or referenced (I3). The cloud backup contains the decoy's Share B — this is both correct (the decoy set should be recoverable too) and deniability-preserving (the cloud share's existence reveals nothing about which set it protects).

8. PIN rotation and share lifecycle

8.1 PIN change (KEK changes, DEK unchanged)

1. Unwrap Share A with old KEK → get Share A plaintext.
2. Re-wrap Share A with new KEK → store.
3. Shares B and C are unaffected (they don't depend on the KEK).
4. Fast-path DEK cache: re-wrap with new KEK.

No Shamir resplit. No cloud update. No QR re-export.

8.2 DEK rotation (rare — vault rekey or share compromise)

If the DEK must change (e.g., vault v:3 migration re-encrypts the seed under a new DEK, or a share is believed compromised):

1. Generate new DEK.
2. Re-encrypt seed under new DEK.
3. Shamir-split new DEK into 3 new shares.
4. KEK-wrap new Share A, store.
5. Recovery-Key-wrap new Share B, push to cloud.
6. Prompt user to re-export Share C (old QR/NFC is invalidated).

This is the expensive path. It should be rare — the design minimizes it by keeping the DEK stable across PIN changes.

8.3 Panic wipe

Panic wipe destroys:

- Share A (cleared from device keystore)
- Fast-path DEK cache (cleared)
- The vault ciphertext itself

Shares B (cloud) and C (paper) survive. This is intentional: if the user is coerced into a panic wipe and later regains safety, they can recover from B+C on a new device. The recovery passphrase is the gate — an attacker who forced the wipe but doesn't know the recovery passphrase cannot use the cloud share.

If panic wipe should be total (no recovery): the user can disable cloud backup before wiping, or the panic-wipe flow can offer a "destroy cloud share" option that calls the platform backup API to delete the entry. This is an owner decision — spec presents both options.

9. Security Posture Assessment (post-onboard trigger)

Recovery setup is **not part of onboarding**. Onboarding stays at 7–8 steps (Welcome → PIN → Explore → Create/Import → Provisioning → KEK → Consent → Tour → Dashboard).

Instead, recovery setup is triggered by a **Security Posture Assessment** — a persistent prompt on the dashboard that appears after onboarding completes. The assessment grades the wallet's resilience and nudges the user toward recovery setup when they have context (funds at risk) rather than during abstract first-run setup.

9.0 Posture levels and scoring

The posture assessment presents a **percentage score** with a graduated colour that shifts from red → amber → green as the user completes each security layer. The score and colour are visible on the dashboard as a persistent indicator (shield icon + arc meter) until the user reaches 100%.

The posture score is a **multi-dimensional security health assessment**, not just a recovery readiness meter. It covers the full security surface — authentication, device integrity, hardware binding, and recovery — so the user sees their wallet's overall protection posture, not only whether their funds are recoverable.

9.0.1a Scoring dimensions

Each dimension contributes independently to the total score. The overall percentage is the weighted sum; the colour is interpolated from the total.

Dimension	Weight	Checks	Max pts
Authentication	20%	PIN created (10) · PIN $\geq$ 12 chars (5) · Biometric/FaceID enrolled (5)	20
Device Integrity (RASP)	25%	RASP scan passed (10) · No root/jailbreak (5) · No Frida/instrumentation (5) · Play Integrity / App Attest valid (5)	25
Hardware Binding	15%	KEK active (5) · Hardware tier StrongBox/SE (5) · TEE accepted, lower score (3)	15
Recovery	30%	Recovery passphrase set (8) · Share A KEK-wrapped (2) · Share B cloud-uploaded (8) · Share C exported (6) · Share C verified (6)	30
Session Security	10%	WalletConnect spend limit set (3) · Session expiry $\leq$ 24h (3) · Step-up re-auth enabled (4)	10

Total: 100 points → percentage. Colour thresholds:

Range	Colour	Label
0–30%	Red (#E85A5A)	Critical
31–50%	Amber (#E8A838)	Weak
51–70%	Yellow (#D4C44A)	Fair
71–85%	Yellow-green (#B8D44A)	Strong
86–100%	Green (#4ADAC2)	Complete

9.0.1b Dashboard banner copy (dimension-aware)

The banner surfaces the **lowest-scoring dimension first**, not a fixed sequence:

- RASP failed (0/25): "Device integrity issue detected — tap to review"
- Auth 10/20: "Enable FaceID to strengthen authentication"
- Hardware 5/15: "Hardware protection available — enable for stronger binding"
- Recovery 0/30: "No recovery — set up backup to protect against device loss"
- Session 7/10: "Tighten WalletConnect session settings"

When multiple dimensions are low, the banner cycles or stacks the top 2 by impact.

9.0.1c RASP integration

The RASP tier (BLOCK/WARN/ALLOW) feeds directly into the Device Integrity dimension:

- **ALLOW (clean):** full 25 points.
- **WARN (anomaly detected):** 10 points. Banner: "Security anomaly detected — some features may be restricted." The posture screen shows which check triggered the warning (root, Frida, tampered APK, missing attestation) without disclosing detection internals (I4).

- **BLOCK (compromised):** 0 points. Banner: "Device integrity check failed." The posture meter is visually locked at the Device Integrity segment — the user sees that this dimension is holding the score down. Recovery setup is still available (the user's funds should still be recoverable from a compromised device), but the posture will not reach 100% until RASP clears.

RASP score is read-only in the posture UI — the user cannot "fix" a root detection by tapping a button. The posture screen explains what was detected and what it means, honestly: "Your device appears to be rooted. This increases the risk of key extraction. Consider migrating to a stock device."

9.0.1d Score persistence and deniability

The posture score is computed live on each dashboard mount — NOT persisted to localStorage (which would leak posture state across credential sets). Each credential set's posture is derived from its own KEK enrollment, share state, and the shared RASP result (RASP is device-level, not credential-level — this is honest; a rooted device is rooted for all credential sets).

In decoy/demo sessions, the posture operates on the decoy's own share/KEK state (I3). The RASP dimension shows the same result (device-level truth). An examiner sees a plausible posture for the decoy wallet.

At 100% the shield turns solid teal (#4ADAC2) and the banner collapses to a single-line "Fully protected" confirmation that can be dismissed permanently.

Degradation: if any dimension drops (biometric changed → KEK invalidated, cloud backup disabled, RASP detects new anomaly), the score recalculates live and the colour reverts. The banner reappears with context: "Your security score dropped — [dimension]: [reason]. Tap to review."

9.0.1 Trigger conditions (when to show the assessment)

The assessment prompt appears when ANY of these conditions is met, checked on each dashboard mount:

1. **First dashboard load** after onboarding completes (immediate, skippable).
2. **First incoming transaction** — the wallet now holds real value; recovery becomes concrete. A modal appears: "You just received [amount]. Set up recovery?"
3. **24 hours after wallet creation** if still unprotected — a one-time notification (not a blocking modal).
4. **Settings → Security** — always available as a manual entry point.

The prompt is **always skippable**. Skipping sets a cooldown (7 days before the next nudge, except the incoming-transaction trigger which fires once regardless of cooldown). After 3 skips the banner remains but modals stop — the user has decided.

9.0.2 Deniability in the assessment

In decoy/demo sessions, the assessment operates on the decoy DEK's posture (I3). The banner text is identical — an examiner sees the same "set up recovery" prompt regardless of which credential set is active. No real-set posture state is read or written.

10. Share C step-by-step workflows

10.1 Export — happy path

Step	Actor	Action	Crypto	Failure mode
1	User	Taps "Set up recovery" from posture assessment or Settings → Security	—	—
2	User	Enters PIN + biometric	—	Auth rejected → retry or abort; no shares created
3	App	KEK unwrap → DEK in memory	<code>unwrapDek(kek, wrappedDek)</code>	KEK wrong → <code>KEK_ERR.UNWRAP_FAILED</code> ; abort
4	App	Shamir-split DEK into 3 shares (fresh polynomial, <code>crypto.getRandomValues</code> for coefficients)	$GF(2^8)$ $k=2, n=3$	RNG failure → throw; no shares persisted

5	App	Display Share C as QR code (or prompt NFC tap / file save)	Base44 encode of 33 bytes (vxrs: prefix)	—
6	User	Re-scans their own QR / re-taps NFC to verify	—	User skips → flow blocked; verification is mandatory
7	App	Byte-compare scanned value against generated Share C	Constant-time compare	Mismatch → "QR damaged or wrong. Re-export." (I4); loop to step 5 with same share bytes
8	App	Persist: Share A → KEK-wrap → Keystore; DEK fast-path cache → KEK-wrap → Keystore; Share B → Recovery-Key-wrap → cloud backup	AES-256-GCM wrap ×3	Keystore write failure → abort, zero all shares in memory; cloud upload failure → warn user, mark Share B as pending (retry on next unlock)
9	App	Show "Recovery active" confirmation	—	—

Invariants enforced throughout:

- No share is persisted until step 7 verification passes.
- If the user closes mid-flow (steps 1–7), no shares exist anywhere — restart is clean.
- All share material is zeroed in a `finally` block after step 8 (success or failure).
- In deniability/demo mode, the flow operates on the decoy DEK's shares (I3).

10.2 Export — unhappy paths

10.2.1 Authentication failure (step 2)

User enters wrong PIN or biometric rejected

- "Incorrect PIN" / "Biometric not recognized"
- Retry (up to lockout threshold)
- No shares generated, no state changed

10.2.2 QR verification mismatch (step 7)

User scans a QR that doesn't match (photographed wrong one, camera misread)

- Constant-time compare fails
- "This QR doesn't match the recovery share. Please scan the one just shown."
- Re-display the SAME QR (same share bytes – do NOT re-split)
- User re-scans
- 3 failed attempts → offer to restart from step 4 (new polynomial)

10.2.3 NFC write failure (step 5, NFC path)

NFC tag too small (< 64 bytes usable) or tag pulled away mid-write

- "NFC write failed. Hold the tag steady and try again."
- Retry write with same share bytes
- After 3 failures → offer QR fallback ("Show as QR code instead?")

10.2.4 Cloud upload failure (step 8, Share B)

Platform backup API unavailable or disabled by user

- Share A and DEK cache are still persisted (local recovery works)
- Warning: "Cloud backup unavailable. Recovery requires your device + QR."
- Share B marked as PENDING in vault metadata
- On next successful unlock, retry cloud upload silently

- If cloud remains unavailable after 7 days, prompt user once
- User is in 2-of-2 mode (A+C) until cloud share lands – honest about this

10.2.5 Keystore write failure (step 8, Share A)

Secure storage full or platform error on write

- ABORT entire flow – do NOT persist partial shares
- Zero all share material in memory
- "Recovery setup failed. Your wallet is unchanged."
- No retry – user must re-initiate from step 1
- Vault continues on the pre-Shamir full-DEK path

10.3 Recovery — happy path (device lost, B + C)

Step	Actor	Action	Crypto	Failure mode
1	User	Installs Veyrnox on new device	—	—
2	App	Detects no local vault; offers "Recover from backup"	—	—
3	Platform	Cloud backup restores encrypted Share B blob to device	Platform E2E decryption (automatic)	Share B absent → §9.4.1
4	User	Enters recovery passphrase	—	Wrong passphrase → §9.4.2
5	App	$\text{Argon2id}(\text{passphrase}, \text{salt}) \rightarrow \text{Recovery Key} \rightarrow \text{AES-256-GCM decrypt Share B}$	Argon2id + AES-GCM	GCM auth failure → "Incorrect passphrase"
6	User	Scans QR code / taps NFC tag / imports .veyrnox-share file	Base44 decode + prefix check	Invalid format → §9.4.3; wrong share index → §9.4.4
7	App	$\text{shamir_combine}(\text{Share_B}, \text{Share_C}) \rightarrow \text{DEK reconstructed}$	$\text{GF}(2^8)$ Lagrange interpolation	— (combine is infallible given 2 valid shares with distinct indices)
8	App	$\text{decryptVaultWithDek}(\text{dek}, \text{vaultCiphertext}) \rightarrow \text{seed in memory}$	AES-256-GCM	GCM auth failure → §9.4.5 (wrong DEK — old share from previous polynomial)
9	User	Sets new PIN; app enrolls new hardware factor H on this device → new KEK; Shamir re-splits DEK → new Share A (KEK-wrapped), re-encrypts Share B (new Recovery Key, new salt), Share C unchanged	HKDF + AES-GCM wrap + $\text{GF}(2^8)$ split	—
10	App	"Wallet restored" confirmation; prompt to re-export Share C (new polynomial invalidates old C)	—	—

Post-recovery state: new device has its own KEK(H_{new} , C_{new}), fresh Share A, fresh Share B in cloud. Old Share C from the pre-recovery polynomial is **invalidated** — user should destroy it and export a new one. The app prompts for this but cannot enforce physical destruction.

10.4 Recovery — unhappy paths

10.4.1 Cloud share missing (step 3)

Platform backup disabled, new Apple/Google account, or cloud data wiped

- "No cloud backup found."
- Recovery requires Share A (old device) + Share C
- If old device is also lost → §9.4.6 (unrecoverable)
- Offer fallback: "Do you have a .enc backup file?" → existing envelope path

10.4.2 Wrong recovery passphrase (step 5)

User enters wrong passphrase → Argon2id derives wrong Recovery Key

- AES-256-GCM decrypt → auth tag mismatch
- "Incorrect passphrase. Please try again."
- No information leaked about HOW wrong (I4 – generic error)
- No attempt limit (Argon2id 192 MiB makes brute-force impractical)
- After 5 consecutive failures → suggest: "Are you sure this is the right passphrase? You can also recover with your device + QR if available."

10.4.3 QR unreadable or invalid format (step 6)

Camera cannot decode QR, or decoded bytes lack VXR: prefix, or length ≠ 33 bytes

- "This doesn't look like a Veyrnox recovery share."
- "Make sure you're scanning the QR code from your recovery setup."
- Retry scan
- After 3 failures → offer file import path ("Import .veyrnox-share file instead?")
- If no valid Share C available → §9.4.6

10.4.4 Duplicate share index (step 6)

User presents a share with the same x-coordinate as Share B (index 2)

- Two shares with index 2 cannot reconstruct the DEK
- "This share has the same index as your cloud share. You need a different recovery share (your QR code or NFC tag)."
- Retry scan – do NOT attempt combine (Shamir requires distinct indices)

10.4.5 DEK reconstruction produces wrong key (step 8)

Shares are from different polynomials (e.g., old Share C + new Share B after a DEK rotation the user forgot about)

- shamir_combine produces a 32-byte value (combine itself never fails)
- AES-256-GCM decrypt of vault ciphertext → auth tag mismatch
- "Recovery failed. Your shares may be from different backup generations."
- "If you rotated your PIN or re-exported shares, you need shares from the same generation."
- ABORT – do NOT retry with same shares (they will produce the same wrong DEK)
- Offer fallback: "Do you have a .enc backup file?"

10.4.6 Unrecoverable — two or more shares lost

Device lost + QR/NFC lost (only cloud share remains)

OR device lost + cloud unavailable (only QR remains)

OR all three lost

- Single share reveals ZERO bits of the DEK (Shamir information-theoretic guarantee)
- "We cannot recover your wallet. With only one recovery share, reconstruction is mathematically impossible."
- "This is a property of non-custodial wallets – no one, including Veyrnox,

```
can override this."
→ Offer: "Create a new wallet" (fresh seed)
→ Honest, final, no false hope (I4)
```

10.5 Recovery — alternative paths

10.5.1 Cloud unavailable, recover from device + local (A + C)

1. Device has Share A (KEK-wrapped). User unlocks normally → Share A.
2. User presents Share C (scan QR / tap NFC).
3. `shamir_combine(Share_A, Share_C)` → DEK reconstructed.
4. This path is primarily for re-establishing cloud Share B after a cloud backup failure or platform migration.
5. App re-encrypts Share B and uploads to cloud.

10.5.2 Paper lost, re-export from device + cloud (A + B)

1. Device has Share A. Cloud has Share B.
2. User unlocks device (PIN + biometric) → Share A.
3. Fetch + decrypt Share B (recovery passphrase).
4. Reconstruct DEK. **Re-split with new polynomial.** Export new Share C.
5. Old Share C (if still physically existing) carries a share from the OLD polynomial — it cannot combine with new shares from the new polynomial. User should destroy old Share C.

Important: re-export MUST generate a new polynomial (new split), not reuse the old one. Otherwise old Share C + new Share C from the same polynomial = 2 shares = DEK. A new split makes old shares from the previous polynomial useless alongside shares from the new one.

10.5.3 Normal unlock (no change)

1. PIN + biometric → KEK → unwrap fast-path DEK cache → decrypt seed.
2. Shamir layer is never invoked.

11. Open questions (owner decisions required)

11.1 Share C deniability under physical search

A printed QR code or NFC tag is physically discoverable. Unlike the encrypted vault blob (which looks like random bytes), a `VXRS:` prefix identifies it as a Veyrnox recovery share. Under coercion, this is evidence that the user has a wallet.

Options:

- (a) Accept this — the backup envelope (`.enc` file) has the same property. Physical backups are opt-in; users who need maximal deniability skip Share C.
- (b) Encode Share C without a prefix — bare Base44, indistinguishable from a random string. Reconstruction tries Shamir combine on any 33-byte input. The cost is worse UX (no type detection on scan; user must manually confirm "this is a recovery share").
- (c) Steganographic encoding — embed the share in an innocuous-looking image or text. Complex, fragile, and the security-through-obscurity is easily broken by a sophisticated examiner. Not recommended.

Recommendation: (a). Document the tradeoff honestly. Users who need coercion-resistance rely on cloud share + device, not physical paper.

11.2 Recovery passphrase vs. existing password

Should the recovery passphrase be the same as the wallet password, or a separate credential?

Options:

- (a) Same password — simpler UX, one fewer thing to remember. But: if the password is compromised, both the vault (if device is seized) and the cloud share are exposed.
- (b) Separate passphrase — defense-in-depth, but users must remember two strong credentials. Most will write them down together, defeating the separation.

- (c) Derived — the recovery key is derived from the wallet password PLUS a recovery-specific salt and domain string, so the same password produces different keys for vault and cloud share. An attacker with the password can derive both, but a brute-force against the cloud share doesn't directly yield the vault key (different salt).

Recommendation: (c). Same user input, cryptographically separated keys, no additional memorization burden.

11.3 Panic wipe and cloud share

Should panic wipe destroy the cloud share?

Options:

- (a) No — the cloud share survives, allowing post-coercion recovery. This is the design's main value proposition for coercion-resistant wallets.
- (b) Yes — total wipe means total wipe. An attacker who coerces a wipe and then obtains Share C later cannot combine B+C.
- (c) User choice at panic setup time — "Should your cloud backup survive a panic wipe?" with honest copy explaining the tradeoff.

Recommendation: (a) as default, with (c) available in advanced settings. The coercion model's value is that the user can appear to have fully complied while retaining a recovery path.

11.4 2-of-2 vs. 2-of-3

Should users who skip Share C (cloud-only backup) get a 2-of-2 split (A+B)?

Options:

- (a) Always 3 shares, Share C stored but unexported (auto-deleted after N days if never exported). Keeps the threshold at 2-of-3; a compromised cloud share alone is still useless.
- (b) Adaptive threshold: 2-of-2 if only A+B, 2-of-3 if C is exported. Simpler for cloud-only users but a secret split of 2-of-2 means each share alone is the full DEK XOR'd — weaker than 2-of-3 information-theoretically.
- (c) Always 2-of-3, but Share C is generated and shown once ("write this down or it's gone"). Mirrors the seed-phrase ceremony.

Recommendation: (c). It matches the mental model users already have from seed phrase backup, and it maintains the full Shamir security property.

11.5 Interaction with existing backup envelope

The backup envelope (`vaultBackup.js`) independently encrypts the full vault container under password and PIN seals. This spec adds Shamir shares of the DEK. Both are recovery mechanisms; they can coexist.

Proposed rule: the backup envelope remains available as-is. Users who want belt-and-braces can export BOTH a `.enc` envelope AND a Share C. The recovery flow tries Shamir first (if shares are detected), then falls back to the envelope. No mechanism is deprecated.

12. Implementation scope

12.1 New modules

Module	Responsibility
<code>src/wallet-core/keystore/shamir.js</code>	GF(2^8) split/combine, constant-time
<code>src/wallet-core/keystore/recoveryShare.js</code>	Share B encryption/decryption, cloud transport
<code>src/wallet-core/keystore/localShare.js</code>	Share C encoding (QR/NFC/file), scanning

12.2 Modified modules

Module	Change
<code>src/wallet-core/keystore/kek.js</code>	None (∅)

src/wallet-core/keystore/native.js	Store Share A + fast-path DEK cache under separate keys
src/wallet-core/vault.js	Vault blob gains shamirIndex field; encryptVaultWithDek / decryptVaultWithDek unchanged
src/wallet-core/keystore/web.js	Platform fence extended to cover share transport
src/components/SecurityPosture.jsx	New dashboard banner + posture assessment (§9)
src/components/RecoverySetup.jsx	New post-onboard flow for passphrase + Share C export
src/components/RecoveryRestore.jsx	New recovery flow

12.3 Test plan

- **Unit (shamir.js):** round-trip split/combine for all 3 pair combinations (A+B, A+C, B+C). Verify that any single share reveals zero bits (statistical test: each byte of a single share is uniformly distributed).
- **Unit (shamir.js):** wrong threshold — combining 1 share produces garbage, not the DEK.
- **Unit (shamir.js):** determinism — same DEK + same polynomial coefficients = same shares (for test reproducibility); different coefficients = different shares (for production freshness).
- **Unit (recoveryShare.js):** round-trip encrypt/decrypt of Share B under a known passphrase. Wrong passphrase → GCM auth failure.
- **Unit (localShare.js):** QR encode/decode round-trip. NFC encode/decode round-trip. Prefix detection.
- **Integration:** full enrollment flow — create wallet, Shamir split, wrap Share A, encrypt Share B, export Share C, wipe, recover from B+C, verify seed matches.
- **Integration:** PIN change — verify Shares B and C are unaffected (same bytes).
- **Integration:** DEK rotation — verify all 3 shares change and old shares from the previous polynomial cannot combine with new shares.
- **Deniability:** enroll decoy set, verify decoy shares are independent of real shares, verify cloud share in decoy session is the decoy's Share B.
- **Panic wipe:** verify Share A and fast-path cache are destroyed; verify Share B survives in cloud (or not, per §11.3 decision).

12.4 Migration path

Existing KEK-enrolled users have a full DEK wrapped by their KEK. Migration:

1. On next unlock, unwrap DEK via existing KEK path.
2. Shamir-split DEK into 3 shares.
3. Re-wrap Share A under KEK (replaces the full-DEK wrap).
4. Store fast-path DEK cache under KEK.
5. Prompt user to set up recovery (passphrase + Share C export).
6. Encrypt and upload Share B to cloud.

Users who decline recovery setup continue with Share A only (functionally identical to today — the fast-path DEK cache means no behavioral change). Share B and C are not created. The vault blob records `shamirEnabled: false` so the app knows to use the full-DEK path.

Non-KEK-enrolled users (PIN-only, no hardware factor) are not affected. This spec requires KEK enrollment as a prerequisite — the recovery passphrase protects Share B, but Share A's security depends on the KEK hardware binding.

13. Audit surface

This spec expands the audit surface by:

1. **Shamir implementation** (~200 LOC) — correct $GF(2^8)$ arithmetic, no off-by-one in polynomial evaluation, constant-time share operations.
2. **Recovery Key derivation** — same Argon2id construction as vault.js, but with a separate domain salt. Verify domain separation is effective.
3. **Cloud transport** — platform backup API usage, encryption-before-upload, no plaintext share in transit.
4. **Share lifecycle** — split, wrap, store, reconstruct, re-split. Each transition must be crash-safe (no partial state).
5. **Deniability** — per-set independence of share sets, no cross-set leakage in cloud storage.

The independent third-party audit (already outstanding for KEK) should cover this layer in the same engagement. The Shamir layer is small enough to audit alongside KEK without a separate engagement.

14. Rejected alternatives

14.1 Shard the seed directly

Rejected: would require re-encrypting the seed on every PIN change (the DEK indirection exists to avoid exactly this). Also complicates the vault format — the vault ciphertext would need to carry share metadata, changing the existing AES-GCM construction.

14.2 MPC (multi-party computation)

Rejected: MPC requires interactive protocols between share-holding parties during signing. Veyrnox is a single-user wallet — there are no other parties. MPC solves a different problem (distributed signing authority) that doesn't apply here.

14.3 Social recovery (Vitalik-style guardians)

Rejected for v1: requires a social graph and a coordination protocol. May be explored as a future extension on top of this Shamir foundation (guardians hold Share C equivalents), but the base layer should work without any social dependencies.

14.4 Cloud-only (no Shamir, just encrypt DEK to cloud)

Rejected: a cloud blob containing the full DEK (even encrypted) means the recovery passphrase alone stands between the cloud provider and the seed. With Shamir, the cloud share alone is information-theoretically zero — a compromised cloud + brute-forced passphrase yields one share, which reveals nothing. The attacker still needs a second share from a different location.

14.5 XOR-based 2-of-2 split

Rejected: XOR splitting ($DEK = S1 \oplus S2$) is simpler but only supports $k=n$ (all shares required). No threshold flexibility, no graceful degradation if one share is lost. Shamir's k -of- n is strictly more capable at negligible complexity cost.

15. Security invariants preserved

Invariant	How this spec preserves it
I1 — keys never leave the device	No plaintext key material leaves the device. Shares are wrapped (AES-GCM or KEK) before export. The share itself is not the key — it is one input to a reconstruction that only happens in-memory on a device.
I2 — no silent data egress	Cloud share upload uses the platform's own backup API (user-configured, user-visible). No custom network calls.
I3 — deniability mode makes zero backend calls	Decoy/demo sessions operate on their own share sets. No real-set shares are read, written, or referenced in deniability mode. Cloud backup in deniability mode backs up the decoy's shares.
I4 — fail honest, fail closed	Shamir combine with <2 shares fails explicitly (<code>SHAMIR_INSUFFICIENT_SHARES</code>). Wrong recovery passphrase fails at GCM auth. No silent fallback to single-share reconstruction.
I5 — backend untrusted by design	The cloud provider sees double-wrapped ciphertext. Even if both encryption layers (Recovery Key + platform E2E) are broken, the result is one Shamir share — zero bits of the DEK.
I6 — Hardware Binding: $KEK = HKDF(H \parallel C)$	Unchanged. The KEK construction is not touched by this spec.

16. Dependency on other work

Dependency	Why	Status
------------	-----	--------

Vault AAD v:3 (#1111)	Share metadata fields should be AAD-bound from day one	PLANNED, owner decision pending
Independent third-party audit	This spec cannot ship before audit review	Outstanding
KEK enrollment on all entry paths	Share A requires KEK; enrollment must be universal	BUILT (PRs #1298, #1301)
Platform backup API access	iOS iCloud Keychain sync, Android Backup API	Not yet implemented

This document is self-contained. It states the problem, the resolved design, the share construction, the recovery flows, the deniability rules, and the open questions, so it stands alone as an audit input and as the brief a Claude Code session builds from.